Audio

All audio is synthesised by the project's Python tools (numpy + scipy, Ogg encoding through the bundled ffmpeg/libvorbis). It is owned by the project and needs no credits. `tools/audio_dsp.py` holds the shared DSP: additive and modal instruments (plucked strings, mallets, bells, reeds, brass, flute, strings, pads, choir, a drum kit), stereo panning and decorrelation, stereo convolution halls, mono-bass and small-speaker bass, a look-ahead limiter, BS.1770 loudness (LUFS) and seamless-loop helpers.

Tool	Output
`tools/gen_music.py`	`assets/audio/music/<song>_base.ogg`, `_action.ogg`, `_climax.ogg`, `sting_*.ogg` (44.1 kHz stereo Vorbis q4)
`tools/gen_ambience.py`	`assets/audio/ambience/bed_<realm>.ogg` (stereo loops), `amb_<detail>_<n>.wav` (stereo one-shots)
`tools/gen_sfx.py`	`assets/audio/sfx/<name>_<n>.wav` (44.1 kHz stereo, variants; Godot imports them as QOA)

Vorbis stores the exact sample count, so looped Ogg files are seamless; `Sound._looped()` turns on looping for Ogg and WAV, so no import settings are needed. Loop seams are checked after encoding: every seam step is smaller than the signal's own 99.9th-percentile sample step.

Runtime mix (`src/autoload/sound.gd`)

Buses: `Master` (glue compressor, then a hard limiter with +3.5 dB pre-gain and a −1.5 dB ceiling) ← `Music` (low-pass), `Ambience` (low-pass), `SFX` (realm reverb), `UI` (dry). The music slider drives `Music`. The sound-effects slider drives `SFX`, `UI` and `Ambience`, so turning music off still leaves the world's sounds. All numbers are in `data/balance.json -> audio`.

Measured with `--record-audio` (Master output, 14 s from launch through the collapse):

Mix	Loudness	Peak	L/R correlation	Impacts over the bed
Level 31, before this upgrade	−21.3 LUFS	−4.4 dBFS	0.89 (nearly mono)	about 10 dB
Level 31, now	−18.2 LUFS	−1.5 dBFS, 0 clipped samples	0.71 (wide)	11.7 dB
Level 12, now	−18.0 LUFS	−1.5 dBFS	0.86	12.6 dB
Home screen, now	−20.6 LUFS	−9.2 dBFS	0.57	music and ambience only
Level 4, playtest fix (2026-10-31), before	−19.8 LUFS	−1.5 dBFS		13.6 dB
Level 4, after	−17.8 LUFS	−1.7 dBFS		19.6 dB
Level 31, after	−18.1 LUFS	−1.8 dBFS		17.0 dB

"Impacts over the bed" = 98th-percentile minus 20th-percentile short-term RMS (0.4 s windows).

Gameplay rule (playtest 2026-10-31: "music and ambience too loud, the focus should be the match SFX"): in aim / flight / destruction the music base stem sits at –10 / –10 / –11 dB, the action stem tops out 12–26 dB down, the energy and climax stems peak at –9 dB, and the ambience bus is –11 dB with a –6 dB action duck. Every block break ducks the music 4 dB, a big impact 5, an explosion 9, a collapse 6; it recovers at 7 dB/s. Menus keep their level (menu ambience offset 0 dB on the lower bus). Target: hits 17 dB or more over the bed.

Positional sound

`Sound.play_at(name, world_pos, vol, pitch)` plays a world sound on an `AudioStreamPlayer2D` from a pool of 12. Godot pans it by where it is on screen and lowers it a little when it is far from the camera. A block breaking on the right of the fortress is heard on the right, and an arrow loosed from a tower on the left is heard on the left. About 35 world call sites use it: impacts, breaks, explosions, fire, quakes, splashes, arrows, archers, mechanisms, the boss rig, captives, floods, Hellfire. UI sounds never become positional. Without a 2D camera (menus), `play_at` behaves like `play`.

Godot's 2D pan law puts a centred source 6.2 dB below a plain player (measured with a test tone through the real bus chain), so `positional.make_up_db` is +6 dB: a centred world sound plays at the same level as before, and off-centre sounds lean left or right. Ambience details (birds, gulls, drips, chimes) also play from random points across the screen and slightly beyond it.

Adaptive music

Every song has three stereo stems of identical length. They are loaded into one `AudioStreamSynchronized`, so they start and loop on the same sample. The mixer sets each stem's volume inside that stream: - **base**: harmony, bass and the lead melody. It always plays. - **action**: drums, ostinato and a melody double. Its level follows the music state. - **climax**: choir, a low brass counter-line, taiko on the downbeats and cymbal swells into every fourth bar. It joins only at the peak of a destruction chain.

State	When	Action stem	Climax stem
`menu` / `map`	home, meta screens, map	off	off
`aim`	a hero is in the bucket	–20 dB, rises with destruction energy	off
`flight`	`Events.projectile_launched`	–10 dB	off
`destruction`	first impact or break after a launch	up to –3 dB	from energy 0.6: –12 dB rising to –2 dB
`result`	`Events.level_finished`	off, base –12 dB under the stinger	off

Destruction energy rises on big impacts (+0.12), broken blocks and defeated archers (+0.18), explosions (+0.5) and chain links (+0.1). It holds for 2.5 s, then decays at 0.22 per second. Drums swell with the carnage. A big collapse brings in the choir and taiko, and the music settles back by itself. Explosions duck the music 5 dB and stingers duck it 12 dB, with a smooth release. Changing song crossfades the two decks over 1.6 s.

Each instrument has its own stereo position (`PAN` in `gen_music.py`). Leads sit near the centre, harp, celesta, bells and hats to the sides, and strings, pads and choir are spread wide. Bass and kick are always centred: everything under 150 Hz is folded to mono, so earbuds and phone speakers get a solid low end. Each stem runs through a stereo convolution hall, and the climax stem's hall is longer. One loudness gain is shared by all stems: base plus action sits at –18 LUFS, and the climax adds about 1 dB on top. Then each stem gets a look-ahead limiter at –1.5 dBFS.

Song	Character
`menu`	Heroic C-major march: horn and flute lead, strings, pizzicato, snare
`blossom`	D yo-scale: koto and shakuhachi, harp arpeggios, taiko and woodblock

Song	Character
`desert`	D phrygian dominant: oud and flute over a reed drone, doum/tek frame drums, tambourine
`harbor`	D dorian 6/8 shanty: accordion, banjo, tuba bass, claps and tambourine
`mines`	A minor: marimba and celesta, low strings, toms and anvil hits
`sky`	F lydian: celesta and flute, harp, airy pad, light kit and bells
`boss`	E harmonic minor at 128 BPM: horn and string ostinato, taiko, brass stabs

The world map plays the music and ambience of the realm in view and crossfades as you scroll. Stingers (stereo, –17.5 LUFS): `win`, `lose`, `boss_intro`, `siege_start`, `reward`.

Surroundings

`Sound.set_surroundings(realm, menu)` crossfades a 24 s stereo ambience bed and scatters detail one-shots at random gaps, levels, pitches and screen positions, so the bed never loops audibly. It also sets the SFX bus reverb for the realm. Beds are rendered once per channel from different noise with the same seeded movement, so gusts and swells move together while the field is wide (L/R correlation about 0.25). In the harbor, the waves roll from one side to the other every 8 s. Menus play the bed 5 dB lower, with one detail type only.

Realm	Bed	Details	SFX reverb
Blossom Valley	breeze, leaves, distant stream, faint crickets	birds, dove	open garden
Desert Ruins	warm wind, whistling gusts, sand hiss	gusts, hawk	dry, long pre-delay
Pirate Harbor	swell and wash in 8 s waves, rolling side to side, lapping water	gulls, rope creaks, ship bell	medium
Crystal Mines	cave room tone, low drone, rumble	water drips, lava bubbles, distant picks	big cave (wet 0.17)
Sky Islands	high wind, airy hiss	wind chimes, gusts, high birds	wide open

Pause and long slow motion glide a low-pass over music and ambience: 750 Hz when paused, 1.5 kHz in slow motion. Slow motion longer than 0.15 s also drops world SFX pitch toward the time scale. Short hit-stops do not.

SFX

Pools: 14 world voices, 12 positional world voices, 4 UI voices, 3 + 3 ambience voices. A quiet sound never steals a loud impact. Each sound has a cooldown and pitch variation from `audio.cooldown_ms` / `pitch_variation`. Names in `audio.ui_sounds` go to the dry UI bus.

The sounds heard most often are built as layer stacks, the way film and console sound designers build them:

Layer	What it is
Transient	a 1–20 ms band-passed crack that gives the hit its edge
Body	modal resonance of the material: damped sine modes of a plank, a stone block or glass. The mode ratios change per variant
Weight	a pitched thump that drops in pitch, plus psychoacoustic bass (below)

Layer	What it is
Texture	granular splinters, grit, shards or debris: dozens of short grains, each panned somewhere across the stereo field
Tail	pieces that bounce with shrinking gaps, rubble roll, dust, and for explosions a darker ground reflection 160–240 ms later, off to one side

Sound	Variants	Built from
`wood_hit`	5	contact click, hollow 5-mode body, low thump, small rattle
`wood_break`	4	crack, fibre tear, 18–27 splinters across the field, 5 tumbling planks, groan
`stone_hit`	5	pitched thud, stony mid body, chunk and grit
`stone_break`	4	crack, 34-grain crumble, rubble roll, 6 bouncing chunks, wide dust
`explosion`	3	snap, chest punch, darkening roar, sub tail, wide roar, 40 debris grains, crackle, ground slap-back
`launch`	3	latch click, rope twang, arm whoosh, crossbar thunk, rattle
`crystal_break` / `ice_break`	4 / 4	glassy crack, inharmonic ring, shards or tinkles spread across the field
`collapse_s/m/l`	2 each	the fall of a tower: chunky low whump, 14/24/36 wood and stone clacks tumbling left to right and thinning out, crumble grains, a darkening dust swell, late pieces
`rubble_settle`	3	pebbles and splinters trickling, one last plank tipping over
`stretch_loop`	loop	a smooth elastic band drawing taut: warm sine harmonics in two slightly detuned voices, a 0.4% slow vibrato, a soft breathy air layer, low-passed at 3.2 kHz (1% of energy above 3 kHz; the old rope creak had 31%)

Collapse layer (after how Angry Birds sells a falling castle)

Per-block break sounds alone make a big fall sound like a pile of separate clicks. Angry Birds treats a collapse as one event: the individual cracks, then a body for the whole fall, then the debris settling. `Sound._on_break` (data `audio.collapse`) groups breaks inside a 700 ms window into one fall. When the window holds 3 / 6 / 10 breaks (or 9 / 20 / 36 mass), the small / medium / large collapse sound plays once at the centre of the breaks and ducks the music. 900 ms after the last break, `rubble_settle` plays. `audio.max_voices` caps each break sound (wood_break 3, stone_break 3, ...), so twenty planks at once don't turn into noise; the collapse layer carries the size instead. `--log-audio` prints each collapse tier.

`gen_sfx.py` runs every sound through `polish()`: - a 40 Hz rumble cut and a top roll-off (12 kHz for the layered set, 8.5–9.5 kHz for the rest); - **small-speaker bass** on impacts and big sounds: harmonics of the sub band are added an octave and a twelfth up. A phone speaker can't play 60 Hz, but the ear still hears the fundamental from those harmonics, so hits keep their weight; - a short stereo room by family (UI nearly dry and narrow; impacts 0.45 s; big sounds 0.9 s at full width); - mono bass under 120 Hz, then soft-knee saturation.

Mastering sets the RMS target on the mid signal and uses a 1 ms fade-in, so the crack of a hit survives. A look-ahead limiter takes up to 3–6 dB off the first transient of the layered set, so the sound is louder at the same peak.

Action	Sound(s)
Grab the catapult / pull	`aim_grab`, looping `stretch_loop` elastic tension tone (`Sound.stretch(power)`: pitch 0.88→1.16, –20→–8 dB), soft `ratchet_0..2` ticks every 0.2 of power

Action	Sound(s)
Bell set-pieces	`bell_0/1` (bronze strike)
Launch / flight	`launch`, `whoosh`
Hero ability	`ability` sparkle + the hero's own `slam` / `fire` / `explosion` / `wind` / `quake`
Wood, stone, ice, crystal, iron hit and break	`wood_*`, `stone_*`, `ice_*`, `crystal_break`, `metal_*`, `straw_break`
Chain reaction	`combo` rising a semitone per link, `star` on milestones, `slowmo` / `slowmo_out`
Mechanisms	`lever`, `gears`, `trapdoor`, `switch_click`, `chain`, `gate_drop`, `rope_snap`, `geyser`
Boss rig	`cannon`, `repair`, `boss_roar`, `metal_bend`, `rune_charge`, `rune_beam`
Units	`arrow_shoot`, `arrow_hit`, `soldier_hit` (armour hit: body thud, metal clank, leather slap; no voice), `soldier_ko`, `enemy_ko`, `enemy_laugh`
Hellfire	`hellfire_rumble` (deep sky rumble at the start), `hellfire_incoming` (falling roar, every few meteors), `explosion` per impact
Story dialogue	`talk_hero`, `talk_high` (wizard, captive), `talk_enemy`: gibberish cartoon voices
Buttons	`ui_tap`, `ui_back` (home), `tab` (side rail), `play_press` (PLAY plank)
Popups / screens	`popup_open`, `popup_close`, `screen_whoosh`
Locked / can't afford	`locked`, `deny`, `toast`
Rewards	`coin_tink`, `gem_tink`, `wallet_land`, `count_tick` (count-ups), `purchase`, `chest_open`, `reveal`, `upgrade`, `anvil`, `wheel_tick`
Map	`map_select`, `locked`

In-level level and regeneration

- `audio.level_sfx_db` (-4 dB) is added to every non-UI sound played through `Sound.play` / `Sound.play_at`.
- `arrow_shoot`, `arrow_hit` and `screen_whoosh` were remade softer (low twang, muffled thump, 0.7 s filtered swell).
- Regenerate only some sounds: `python3 tools/gen_sfx.py --only screen_whoosh,arrow_shoot`.

Tool evaluation

The owner asked for these tools to be used if they are better than what the game had. Each was judged on sound quality on a phone, CPU cost on mid-range Android, APK size, licence and how well it fits a GDScript-only Godot 4.7 project with an offline generator pipeline.

Tool	Verdict	Why
`AudioStreamSynchronized`	**Used**	Sample-locked music stems in one stream. It replaced two separately started players per song and made the third (climax) stem cheap to add.
`AudioStreamInteractive`	Not used	Built for clip-to-clip transitions (intro → loop → outro) on beat. The game's music changes by layering inside one loop. Song changes are crossfades, which the two-deck mixer already does.
`AudioStreamPolyphonic`	Not used	Fewer nodes for many short voices, but no per-voice positional panning. The pooled `AudioStreamPlayer2D` gives

Tool	Verdict	Why
		positional sound and voice stealing.
`AudioStreamPlayer2D`	**Used**	Screen-position panning and distance attenuation for world sounds.
`AudioStreamGenerator`	Not used	Runtime synthesis in GDScript runs on the main thread per audio block. On a phone it costs CPU and risks crackle under physics load. All synthesis is done offline in Python with far more DSP than a runtime budget allows.
Buses, LPF, HPF, BPF, `AudioEffectReverb`, compressor, `AudioEffectHardLimiter`	**Used**	Pause and slow-motion low-pass, per-realm reverb on SFX, glue compressor and limiter on Master (now with make-up gain).
GDSiON (FM synth / MML, GDExtension)	Not used	Native libraries for each Android ABI and runtime synthesis cost. Its FM/chiptune character doesn't suit the glossy cartoon-orchestral style.
GodotSfxr	Not used	sfxr makes retro 8-bit effects. The layered modal/granular recipes in `gen_sfx.py` go well past it for this style.
Godot Synth Engine	Not used	Runtime synthesis, same CPU argument as `AudioStreamGenerator`. Pre-rendered audio costs nothing at runtime.
LibPD (Pure Data)	Not used	A full patch runtime and native library for a game whose sounds are fixed one-shots and loops. It suits procedural sound that reacts to many parameters, which this game doesn't need.
Sonilo	Not evaluated	I couldn't identify a maintained Godot 4 plugin by this name, so it was not installed.
FMOD / Wwise GDExtensions	Not used	They need the authoring tool workflow, native libraries for each ABI (several MB), registration, and licence terms with budget/revenue limits. Godot 4.7's built-ins now cover what the game uses: synced stems, buses, effects and 2D panning. Revisit if the game ever needs hundreds of simultaneous voices or a dedicated sound team.
Vital, Surge XT (free synths), Valhalla Supermassive (free reverb)	Optional, offline	These are DAW plugins, not runtime libraries, and none is installed on the dev Mac. A composer can render stems with them and drop the files in as `<song>_base/action/climax.ogg` with equal length. `Sound` and the tests accept them unchanged. The stereo halls in `audio_dsp.py` do the job Supermassive would do here.
Audacity / Reaper	Optional, offline	Audacity 4 is installed. Use it to audition, trim or loudness-check files (target: base + action at –18 LUFS, stingers at –17.5 LUFS).

Checking the mix without listening

Run `$GODOT --path . -- --level=31 --autoshot=38,0.85,0.9 --record-audio=14[,path]` (debug builds). It writes the Master output to `build/shots/audio.wav` (or the given path), so loudness, peaks, stereo width and spectrum can be measured, for example with `pyloudnorm`. `tests/run_tests.gd` (`_audio_tests`) checks: - every `Sound.play` /

`Sound.play_at` name has a file; - every song has three equal-length looping Ogg stems inside an `AudioStreamSynchronized`; - stingers exist and every realm has its ambience; - the event-driven state machine moves through its states; - the climax stem stays silent at low energy and rises at peak energy; - `play_at` uses a positional voice at the world position, and UI sounds never do.

Studio intro audio

`tools/capture_intro.py` records the intro and then runs `tools/master_intro_audio.py`. Most of the score's energy sits below 150 Hz, which phone speakers can't play, so it sounded far quieter than the menu music. The master step adds harmonics above the bass, trims sub rumble, sets the level above 300 Hz to –27 dB (`TARGET_DB`, about the menu music), then rides the level so no 250 ms stretch goes above RMS 0.30 (`RMS_CEIL`) and peak-limits at –1 dBFS with a smooth 5 ms / 120 ms gain limiter. The older per-sample tanh limiter sat on the loud INTERCORE hit for 4 s (RMS 0.63–0.74) and its waveshaping crackled on phone speakers; the STUDIOS part was already below the ceiling.

Possible later work

- Recorded or licensed instruments for a final soundtrack (list them in `docs/CREDITS.md`). They can be dropped in as the same three stems.